

Pontifícia Universidade Católica do Rio Grande do Sul

Disciplina de Sistemas Operacionais

T1

Manual do Usuário

Integrantes: Luiz Augusto Guerra
João Francisco
Humberto Corrêa Gomes
Gustavo Munoz

Professor(a): Miguel Xavier

Porto Alegre, 14 de maio de 2026

1. Pré-requisitos

- Python 3.8 ou superior
- Nenhuma biblioteca externa necessária

Verifique sua versão do Python:

```
python --version
```

2. Estrutura do Projeto

```
tp1-so/  
|-- main.py           # Ponto de entrada - executa o simulador  
|-- processo.py       # Classe que representa um processo  
|-- parser_asm.py     # Le e interpreta arquivos .asm  
|-- executor.py       # Executa instrucoes do assembly  
|-- escalonador.py    # Politica de escalonamento EDF  
|-- simulador.py      # Loop principal da simulacao  
'-- programas/       # Pasta com os programas .asm de exemplo  
    |-- p1.asm        # Contador regressivo com impressao na tela  
    '-- p2.asm        # Contador com loop finito, sem bloqueio
```

3. Como Executar

Na raiz do projeto, rode:

```
python main.py
```

O programa irá solicitar, para cada processo:

- **Nome** – Identificador do processo (ex: P1).
- **Arquivo .asm** – Caminho do programa assembly (ex: programas/p1.asm).
- **Arrival time** – Instante de chegada na simulação (ex: 0).
- **Ci** – Tempo de computação em unidades de tempo (ex: 5).
- **Pi** – Período, igual ao *deadline* (ex: 10).

Quando terminar de cadastrar os processos, digite **iniciar** para começar a simulação.

4. Programas de Exemplo

programas/p1.asm

Contador regressivo que imprime na tela e usa **SYSCALL** (causa bloqueio).

```
.code  
LOAD variable  
ponto1: SUB #1
```

```
SYSCALL 1
BRPOS ponto1
SYSCALL 0
.endcode
.data
variable 3
.endcode
```

O que faz: carrega o valor 3, subtrai 1 a cada iteração, imprime o resultado e repete enquanto o acumulador for positivo. Ao chegar em 0, encerra.

Comportamento esperado: o processo bloqueia a cada SYSCALL 1 por 1 a 3 unidades de tempo.

programas/p2.asm

Contador decrescente sem bloqueio. Ideal para testar preempção pura do EDF.

```
.code
LOAD contador
ponto2: SUB #1
STORE contador
BRPOS ponto2
SYSCALL 0
.endcode
.data
contador 5
.enddata
```

O que faz: conta de 5 até 0 armazenando o valor na memória a cada passo. Encerra ao atingir 0.

Comportamento esperado: nunca bloqueia – só é interrompido por preempção do EDF.

5. Como Criar um Programa .asm

Um programa válido deve ter duas seções obrigatórias:

```
.code
    ; instrucoes aqui
.endcode
.data
    ; variaveis aqui
.enddata
```

Instruções disponíveis

Aritméticas – operam sobre o acumulador (acc):

- ADD op – acc = acc + op

- SUB op – $acc = acc - op$
- MULT op – $acc = acc * op$
- DIV op – $acc = acc / op$ (inteira)

Memória:

- LOAD op – $acc = op$
- STORE var – $var = acc$ (somente modo direto)

Salto:

- BRANY label – Sempre salta
- BRPOS label – Salta se $acc > 0$
- BRZERO label – Salta se $acc = 0$
- BRNEG label – Salta se $acc < 0$

Sistema:

- SYSCALL 0 – Encerra o processo
- SYSCALL 1 – Imprime acc na tela (bloqueia 1–3 unidades)
- SYSCALL 2 – Lê inteiro do teclado (bloqueia 1–3 unidades)

Modos de endereçamento

- **Imediato** (#valor): usa o número diretamente. Ex: ADD #5
- **Direto** (nome): usa o valor da variável. Ex: ADD contador

Labels

Defina um *label* colocando um nome seguido de : antes da instrução:

```
loop: SUB #1
      BRPOS loop
```

6. Exemplo de Sessão Completa

```
python main.py

=====
SIMULADOR EDF - Sistemas Operacionais
=====

--- Cadastrar novo processo ---
Nome do processo (ou 'iniciar' para começar): P1
Caminho do arquivo .asm de P1: programas/p1.asm
Arrival time (instante de chegada): 0
Ci (tempo de computacao): 5
```

```

Pi (periodo = deadline): 10
P1 cadastrado. Memoria: 6 posicao(oes).

--- Cadastrar novo processo ---
Nome do processo (ou 'iniciar' para começar): P2
Caminho do arquivo .asm de P2: programas/p2.asm
Arrival time (instante de chegada): 2
Ci (tempo de computacao): 3
Pi (periodo = deadline): 6
P2 cadastrado. Memoria: 6 posicao(oes).

--- Cadastrar novo processo ---
Nome do processo (ou 'iniciar' para começar): iniciar

Iniciando simulacao com 2 processo(s)...

t=0
  CPU: P1
  [CHEGADA] P1 chegou (deadline=10)
  [ESCALONA] P1 começa a rodar (deadline=10)

t=2
  CPU: P2
  [CHEGADA] P2 chegou (deadline=8)
  [PREEMPÇÃO] P1 preemptado por P2
  [ESCALONA] P2 começa a rodar (deadline=8)
...

```

7. Saída da Simulação

Cada unidade de tempo exibe:

- CPU: <nome> – processo em execução naquele tick
- [CHEGADA] – processo adicionado à fila de prontos
- [ESCALONA] – processo começa a usar a CPU
- [PREEMPÇÃO] – processo de maior prioridade assume a CPU
- [SYSCALL 1] – valor impresso pelo processo
- [BLOQUEIO] – processo aguardando retorno de I/O
- [DESBLOQUEADO] – processo volta à fila de prontos
- [PERÍODO] – processo completou seu Ci e inicia novo período
- [FIM] – processo encerrou com SYSCALL 0
- DEADLINE PERDIDO – processo ultrapassou seu *deadline*

Ao final, é exibido o **Diagrama de Gantt**:

DIAGRAMA DE GANTT

```
P1 |##..#.#  
P2 |..###.#  
   |01234567
```

= processo rodando | . = processo aguardando ou bloqueado